利用并行和函数式编程提升计算效率

Posted on 2023-09-24 17:53:59 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2Vq2MDKP4P3VrAz1NuLP5DJOUIT)

简介

前文介绍了 `async` 用法，很多朋友说用的不多，我对 `async` 的理解就是开辟一个一次性的线程执行并行任务，主线程可以通过 `future` 在合适的时机执行等待汇总结果。本文通过并行和函数式编程，演示快速排序提升效率的一种方式。

快速排序

快速排序（Quick Sort）是一种高效的排序算法，采用分治法的思想进行排序。以下是快速排序的基本步骤：

1. 选择一个基准元素（pivot）：从数组中选择一个元素作为基准元素。选择基准元素的方式有很多种，常见的是选择数组的第一个元素或最后一个元素。
2. 分区（partitioning）：重新排列数组，把比基准元素小的元素放在它的左边，把比基准元素大的元素放在它的右边。在这个过程结束时，基准元素就处于数组的最终位置。
3. 递归排序子数组：递归地对基准元素左边和右边的两个子数组进行快速排序。

以下是一个基本的快速排序的C++实现：

```cpp
//c++ 版本的快速排序算法
template<typename T>
void quick_sort_recursive(T arr[], int start, int end) {
    if (start >= end) return;
    T key = arr[start];
    int left = start, right = end;
    while(left < right) {
        while (arr[right] >= key && left < right) right--;
        while (arr[left] <= key && left < right) left++;
        std::swap(arr[left], arr[right]);
    }

    if (arr[left] < key) {
        std::swap(arr[left], arr[start]);
    }


    quick_sort_recursive(arr, start, left - 1);
    quick_sort_recursive(arr, left + 1, end);
}

template<typename T>
void quick_sort(T arr[], int len) {
    quick_sort_recursive(arr, 0, len - 1);
}
```

排序演示

假设一开始序列{xi}是：5，3，7，6，4，1，0，2，9，10，8。

此时，ref=5，i=1，j=11，从后往前找，第一个比5小的数是x8=2，因此序列为：2，3，7，6，4，1，0，5，9，10，8。

此时i=1，j=8，从前往后找，第一个比5大的数是x3=7，因此序列为：2，3，5，6，4，1，0，7，9，10，8。

此时，i=3，j=8，从第8位往前找，第一个比5小的数是x7=0，因此：2，3，0，6，4，1，5，7，9，10，8。

此时，i=3，j=7，从第3位往后找，第一个比5大的数是x4=6，因此：2，3，0，5，4，1，6，7，9，10，8。

此时，i=4，j=7，从第7位往前找，第一个比5小的数是x6=1，因此：2，3，0，1，4，5，6，7，9，10，8。

此时，i=4，j=6，从第4位往后找，直到第6位才有比5大的数，这时，i=j=6，ref成为一条分界线，它之前的数都比它小，之后的数都比它大，对于前后两部分数，可以采用同样的方法来排序。

调用比较简单

```
1. void test_quick_sort() {
2.
3.     int num_arr[] = { 5,  3,  7,  6,  4,  1,  0,  2,  9,  10,  8 };
4.     int length = sizeof(num_arr) / sizeof(int);
5.      quick_sort(num_arr, length );
6.     std::cout << "sorted result is ";
7.     for (int i = 0; i < length; i++) {
8.         std::cout << " " << num_arr[i];
9.     }
10.
11.    std::cout << std::endl;
12. }
```

这种实现方式比较依赖存储数据的数据结构，比如上面是通过数组存储的，那如果我想实现 `list` 容器中元素的排序怎么办？我既不想关注存储的容器，也不想关注存储的类型，想实现一套通用的比较规则？那就需要函数式编程来解决

函数式编程

C++函数式编程是一种编程范式，它将计算视为数学上的函数求值，并避免改变状态和使用可变数据。在函数式编程中，程序是由一系列函数组成的，每个函数都接受输入并产生输出，而且没有任何副作用。

在C++中，函数式编程可以使用函数指针、函数对象（functor）和lambda表达式等机制来实现。这些机制允许您编写可以像普通函数一样调用的代码块，并将它们作为参数传递给其他函数或作为返回值返回。

C++11引入了一些新功能，如constexpr函数和表达式模板，这些功能使得在C++中进行函数式编程更加容易和直观。

我们用函数式编程修改上面的快速排序

```
template<typename T>
std::list<T> sequential_quick_sort(std::list<T> input)
{
    if (input.empty())
    {
        return input;
    }
    std::list<T> result;

    // ① 将input中的第一个元素放入result中，并且将这第一个元素从input中删除
    result.splice(result.begin(), input, input.begin());

    // ② 取result的第一个元素，将来用这个元素做切割，切割input中的列表。
    T const& pivot = *result.begin();

    // ③std::partition 是一个标准库函数，用于将容器或数组中的元素按照指定的条件进行分区，
    // 使得满足条件的元素排在不满足条件的元素之前。
    // 所以经过计算divide_point指向的是input中第一个大于等于pivot的元素
        auto divide_point = std::partition(input.begin(), input.end(),
            [&](T const& t) {return t < pivot; });

    // ④ 我们将小于pivot的元素放入lower_part中
    std::list<T> lower_part;
    lower_part.splice(lower_part.end(), input, input.begin(),
        divide_point);

    // ⑤我们将lower_part传递给sequential_quick_sort 返回一个新的有序的从小到大的序列
    //lower_part 中都是小于divide_point的值
        auto new_lower(
            sequential_quick_sort(std::move(lower_part)));
    // ⑥我们剩余的input列表传递给sequential_quick_sort递归调用，input中都是大于divide_point的值。
        auto new_higher(
            sequential_quick_sort(std::move(input)));
        //⑦到此时new_higher和new_lower都是从小到大排序好的列表
        //将new_higher 拼接到result的尾部
        result.splice(result.end(), new_higher);
        //将new_lower 拼接到result的头部
        result.splice(result.begin(), new_lower);
        return result;
}
```

用如下方式调用

```
1. void test_sequential_quick() {
2.     std::list<int> numlists = { 6,1,0,7,5,2,9,-1 };
3.     auto sort_result = sequential_quick_sort(numlists);
4.     std::cout << "sorted result is ";
5.     for (auto iter = sort_result.begin(); iter != sort_result.end(); iter++) {
6.         std::cout << " " << (*iter);
7.     }
8.     std::cout << std::endl;
9. }
```

这个函数是一个使用快速排序算法对链表进行排序的实现。快速排序是一种常用的排序算法，它的基本思想是选择一个基准元素，然后将数组分为两部分，一部分是小于基准元素的元素，另一部分是大于基准元素的元素。然后对这两部分再分别进行快速排序。这个函数使用了C++的模板，可以处理任何数据类型的链表。函数的主要步骤包括：

1. 将链表的第一个元素作为基准元素，并将其从链表中删除。
2. 使用std::partition函数将链表分为两部分，一部分是小于基准元素的元素，另一部分是大于或等于基准元素的元素。
3. 对这两部分分别进行递归排序。\n4. 将排序后的两部分和基准元素合并，返回排序后的链表。

并行方式

我们提供并行方式的函数式编程，可以极大的利用cpu多核的优势，这在并行计算中很常见。

```
1. //并行版本
2. template<typename T>
3. std::list<T> parallel_quick_sort(std::list<T> input)
4. {
5.     if (input.empty())
6.     {
7.         return input;
8.     }
9.     std::list<T> result;
10.    result.splice(result.begin(), input, input.begin());
11.    T const& pivot = *result.begin();
12.    auto divide_point = std::partition(input.begin(), input.end(),
13.        [&](T const& t) {return t < pivot; });
14.    std::list<T> lower_part;
15.    lower_part.splice(lower_part.end(), input, input.begin(),
16.        divide_point);
17.    // ①因为lower_part是副本，所以并行操作不会引发逻辑错误，这里可以启动future做排序
18.    std::future<std::list<T>> new_lower(
19.        std::async(&parallel_quick_sort<T>, std::move(lower_part)));
20.
21.    // ②
22.    auto new_higher(
23.        parallel_quick_sort(std::move(input)));
24.        result.splice(result.end(), new_higher);
25.        result.splice(result.begin(), new_lower.get());
26.        return result;
27. }
```

测试调用如下

```
1. void test_sequential_quick() {
2.     std::list<int> numlists = { 6,1,0,7,5,2,9,-1 };
3.     auto sort_result = sequential_quick_sort(numlists);
4.     std::cout << "sorted result is ";
5.     for (auto iter = sort_result.begin(); iter != sort_result.end(); iter++) {
6.         std::cout << " " << (*iter);
7.     }
8.     std::cout << std::endl;
9. }
```

我们对 `lower_part` 的排序调用了 `std::async` 并行处理。而 `higher_part` 则是串行执行的。这

么做提高了计算的并行能力，但有人会问如果数组的大小为1024，那么就是2的10次方，需要启动10个线程执行，这仅是对一个1024大小的数组的排序，如果有多个数组排序，开辟线程会不会很多？其实不用担心这个，因为 `async` 的实现方式在上一节中已经提及了，是通过 `std::launch::async` 或者 `std::launch::deffered` 完成的。编译器会计算当前能否开辟线程，如果能则使用 `std::launch::async` 模式开辟线程，如果不能则采用 `std::launch::deffered` 串行执行。当然，我们也可以通过我们上文提及的线程池方式实现并行计算

ThreadPool方式的并行排序

```
1. //线程池版本
2. //并行版本
3. template<typename T>
4. std::list<T> thread_pool_quick_sort(std::list<T> input)
5. {
6.     if (input.empty())
7.     {
8.         return input;
9.     }
10.     std::list<T> result;
11.     result.splice(result.begin(), input, input.begin());
12.     T const& pivot = *result.begin();
13.     auto divide_point = std::partition(input.begin(), input.end(),
14.         [&](T const& t) {return t < pivot; });
15.     std::list<T> lower_part;
16.     lower_part.splice(lower_part.end(), input, input.begin(),
17.         divide_point);
18.     // ①因为lower_part是副本，所以并行操作不会引发逻辑错误，这里投递给线程池处理
19.     auto new_lower = ThreadPool::commit(&parallel_quick_sort<T>, std::move(lower_part));
20.     // ②
21.     auto new_higher(
22.         parallel_quick_sort(std::move(input)));
23.     result.splice(result.end(), new_higher);
24.     result.splice(result.begin(), new_lower.get());
25.     return result;
26. }
```

通过如下方式测试

```
1. void test_thread_pool_sort() {
2.     std::list<int> numlists = { 6,1,0,7,5,2,9,-1 };
3.     auto sort_result = thread_pool_quick_sort(numlists);
4.     std::cout << "sorted result is ";
5.     for (auto iter = sort_result.begin(); iter != sort_result.end(); iter++) {
6.         std::cout << " " << (*iter);
7.     }
8.     std::cout << std::endl;
9. }
```

到此我们实现了多种版本的快速排序，并不是鼓励读者造轮子，而是提供一种并行处理的思想，相信读者在后续的工作中在合适的时机采用并行处理的方式，可以极大的提高程序处理问题的效率。

总结

本文介绍了如何使用future, promise以及async用法

视频链接

https://space.bilibili.com/271469206/channel/collectiondetail?sid=1623290 (https://space.bilibili.com/271469206/channel/collectiondetail?sid=1623290)

源码链接

https://gitee.com/secondtonone1/boostasio-learn (https://gitee.com/secondtonone1/boostasio-learn)

浏览(2109) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) ♡喜欢(521) (JavaScript:void(0);)

热门评论

H B I

请输入正文

评论

没有更多评论了,点击收起 ≪ (JavaScript:void(0);)

热门文章

slice介绍和使用

♡喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

♡喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

♡喜欢(587) 浏览(1595)

解密定时器的实现细节

♡喜欢(566) 浏览(1449)

Linux环境搭建和编码

♡喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站

个人微信



个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。
京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)
京ICP备2021033081号 (https://beian.miit.gov.cn/)